TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 4: Memory

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 4 Topics (5 Hours | 8 Marks):

- 4.1 Memory Write Ability and Storage Permanence
- 4.2 Types of Memory: ROM, PROM, EPROM, EEPROM, Flash, SRAM, DRAM, SDRAM
- 4.3 Composing Memory — Width Expansion and Depth Expansion
- 4.4 ROM Design from Truth Table
- 4.5 Memory Hierarchy
- 4.6 Cache Memory — Mapping Techniques, AMAT, Replacement Algorithms

Chapter 4: Memory

Q1. Explain memory write ability and storage permanence of memory. Show them for different memory types.	2082 Kartik (Back) | 2079 Ashwin (Back) | 2078 Baishakh (Back) | 2075 Baisakh | 2074 Bhadra [3+5]

ANSWER

Write Ability:

Write ability refers to how easily and how often data can be written into a memory device. It ranges from 'write never' (mask ROM) to 'write anytime' (SRAM/DRAM).

Storage Permanence:

Storage permanence describes how long a memory retains its data without power. It ranges from permanent (decades without power) to volatile (data lost immediately on power-off).

Memory Type	Write Ability	Storage Permanence	Volatile?	Typical Use
Mask ROM	Never (factory only)	Permanent (decades)	No	Fixed firmware
PROM	Write once (fuse)	Permanent after programming	No	Field programming
EPROM	UV erase then write (slow)	Very high (10–40 years)	No	Prototyping
EEPROM	Byte-erasable electrically	High (~10 yrs, 100K cycles)	No	Config/calibration data
Flash (NOR)	Block-erasable electrically	High (~10 yrs, 100K cycles)	No	Program storage (code)
Flash (NAND)	Block-erasable electrically	High (similar to NOR)	No	Mass storage (SD cards)
SRAM	Read/Write freely	Volatile (lost on power-off)	Yes	Cache, MCU internal RAM
DRAM	Read/Write (needs refresh)	Volatile (lost on power-off)	Yes	Main memory (PCs, SBCs)

Q2. Explain different types of memory used in embedded systems. Compare SRAM and DRAM.	2079 Jestha (Back) | 2072 Magh (Back) | 2072 Ashwin (Back) [4–8]

ANSWER

Types of Memory:

1. ROM (Read-Only Memory): Data permanently programmed at manufacturing. Used for bootloaders, lookup tables. Cannot be altered.

2. PROM (Programmable ROM): Blank chip programmed once using a hardware programmer by blowing internal fuses. Permanent after programming.

3. EPROM (Erasable PROM): Erased by UV light through quartz window (~20 minutes), then reprogrammable. Used in development.

4. EEPROM: Byte-level electrical erase and reprogram (~100K cycles, ~5ms/byte). Used for non-volatile configuration and calibration storage.

5. Flash Memory (NOR): Block-erased, electrically reprogrammable. Random read access supports code execution in place (XIP). Used as program memory in embedded MCUs.

6. Flash Memory (NAND): Higher density than NOR. Block access only (no random byte read). Used in SD cards, eMMC, SSDs.

7. SRAM (Static RAM): 6-transistor bistable flip-flop per bit. No refresh needed. Fast (2–10 ns). Volatile. Used in caches and MCU internal RAM.

8. DRAM (Dynamic RAM): 1-transistor + 1-capacitor per bit. Requires periodic refresh. High density, low cost. Used as main memory.

9. SDRAM / DDR SDRAM: DRAM synchronized to system clock. DDR transfers on both clock edges. Used in embedded Linux systems.

SRAM vs DRAM:

Property	SRAM	DRAM
Cell structure	6 transistors (flip-flop)	1 transistor + 1 capacitor
Refresh required	No	Yes (every ~64 ms)
Access speed	Very fast (2–10 ns)	Slower (50–70 ns)
Density	Low (large cell)	High (small cell)
Cost per bit	High	Low
Power	Lower active	Higher (refresh current)
Typical use	Cache, MCU internal RAM	Main memory (PCs, SBCs)

Q3. Explain storage permanence of memory. Construct a ROM/PROM to store the data shown: XY=00: F1=0,F2=1,F3=0,F4=0 | XY=01: F1=0,F2=1,F3=1,F4=1 | XY=10: F1=1,F2=1,F3=0,F4=1 | XY=11: F1=1,F2=0,F3=0,F4=1	**2080 Chaitra (Regular) | 2075 Bhadra (Back) [3+5]**

ANSWER

Storage Permanence: (See Q1 table for all types.)

ROM Construction (4×4 ROM — 2-address inputs, 4 outputs):

X	Y	F1	F2	F3	F4
0	0	0	1	0	0
0	1	0	1	1	1
1	0	1	1	0	1
1	1	1	0	0	1

```
2-to-4 Address Decoder:                    Bit Lines
  XY=00 → Word Line 0 (WL0) —— F1=0  F2=1● F3=0  F4=0
  XY=01 → Word Line 1 (WL1) —— F1=0  F2=1● F3=1● F4=1●
  XY=10 → Word Line 2 (WL2) —— F1=1● F2=1● F3=0  F4=1●
  XY=11 → Word Line 3 (WL3) —— F1=1● F2=0  F3=0  F4=1●

  ● = diode/connection present → bit line pulled HIGH = logic 1
    = no connection → bit line LOW via pull-down = logic 0

  Total cells: 4 word lines × 4 bit lines = 16 cells
  Connections present: 8 cells (where stored bit = 1)
```

Structure: A 2-to-4 decoder drives 4 word lines. Each word line connects to 4 bit lines via programmable cells (diodes in PROM). When a word line goes HIGH, it drives connected bit lines HIGH through diodes.

Q4. Compose 2K×16 ROM using 1K×4 ROM chips. Show the full connection diagram.	**2082 Kartik (Back) | 2081 Chaitra (Regular) | 2078 Baishakh (Back) | 2075 Baisakh | 2073 Magh [3+5 / 4+4]**

ANSWER

Analysis:

1. Target: 2K locations × 16 bits/word. Available: 1K locations × 4 bits/word.
2. Width expansion: 16 ÷ 4 = 4 chips in parallel (each contributes 4 bits of the 16-bit word).
3. Depth expansion: 2K ÷ 1K = 2 banks (each bank covers half the address space).
4. Total chips = 4 (width) × 2 (depth) = 8 chips.
5. Address: 2K = 2^11 → 11 bits (A0–A10). Each 1K chip uses 10 bits (A0–A9). A10 selects bank.

```
Address bus: A10  A9...A0
              |    └──── 10-bit row address to ALL chips
              |
         1-to-2 Decoder
            /       \
         CS_0       CS_1
          |           |
     ┌────┴────┐   ┌────┴────┐
     |   BANK 0  |   |   BANK 1  |
     |(A10=0)    |   |(A10=1)    |
     |           |   |           |
   Chip1  Chip2  |   Chip5  Chip6 |   ← D3:D0  D7:D4
   Chip3  Chip4  |   Chip7  Chip8 |   ← D11:D8 D15:D12
     └──────────┘   └──────────┘


All 4 chips in each bank share A0-A9.
Each bank outputs 16 bits (4 chips × 4 bits each).
Only the active bank's CS is asserted by the decoder.
```

Q5. Compose 4K×16 ROM using 1K×8 ROM chips. Also: Compose 4K×32 using 2K×16 chips.

2076 Baisakh (Back) | 2075 Magh (Back) [5+3 / 4+4]

ANSWER

A) Compose 4K×16 ROM using 1K×8 ROM Chips:

1. Target: 4K×16. Available: 1K×8.
2. Width: 16÷8 = 2 chips per bank (in parallel).
3. Depth: 4K÷1K = 4 banks.
4. Total chips = 2×4 = 8 chips.
5. Address: 12 bits (A0–A11). Each chip uses A0–A9. A11–A10 → 2-to-4 decoder → CS0–CS3.

```
A11 A10 | Bank selected | Address range
 0   0  | Bank 0        | 0x000-0x3FF
 0   1  | Bank 1        | 0x400-0x7FF
 1   0  | Bank 2        | 0x800-0xBFF
 1   1  | Bank 3        | 0xC00-0xFFF

Each bank: Chip_High (D15:D8) + Chip_Low (D7:D0)  [both share A0-A9]
```

B) Compose 4K×32 ROM using 2K×16 ROM Chips:

1. Width: 32÷16 = 2 chips per bank.
2. Depth: 4K÷2K = 2 banks.
3. Total chips = 2×2 = 4 chips.
4. Address: 12 bits. Each chip uses A0–A10. A11 → 1-to-2 decoder → CS0 or CS1.

Q6. Construct 2^(k+1) × n memory using 2^k × n memory modules. Also: 2^(k+2) × n using 2^k × n.

2080 Ashwin (Back) | 2081 Ashwin (Back) [4+4]

ANSWER

A) Compose 2^(k+1) × n using 2^k × n modules:

1. Depth ratio = 2^(k+1) / 2^k = 2 → need 2 banks.
2. Width unchanged (n bits) → 1 chip per bank.
3. Total chips = 2.
4. Extra address bit = 1 (bit A_k selects bank via 1-to-2 decoder).

```
Address: A_k  |  A_(k-1)...A_0
          |           |
     1-to-2 dec    Row address (to both chips)
       /     \
    CS_0     CS_1
  Bank 0   Bank 1   (each is 2^k × n bits)
```

B) Compose 2^(k+2) × n using 2^k × n modules:

1. Depth ratio = 2^(k+2) / 2^k = 4 → need 4 banks.
2. Width unchanged → 1 chip per bank.
3. Total chips = 4.
4. Extra address bits = 2 (bits A_(k+1), A_k → 2-to-4 decoder → CS0–CS3).

```
Address: A_(k+1) A_k | A_(k-1)...A_0
          └──────┬──────┘       |
         2-to-4 dec    Row address
           |
    CS0  CS1  CS2  CS3
  Bank0 Bank1 Bank2 Bank3
```

Q7. Compose 2K×16 ROM using 1K×4 ROMs. Determine the better cache: Cache A: 4KB, Miss Rate=15%, Hit Cost=3, Miss Cost=25 Cache B: 12KB, Miss Rate=5%, Hit Cost=6, Miss Cost=25

2078 Baishakh (Back) [5+3]

ANSWER

Composing 2K×16 ROM using 1K×4 ROMs: (See Q4 above — 8 chips, 4-width×2-depth, A10 as bank-select.)

Cache AMAT Comparison:

Formula: AMAT = Hit Cost + Miss Rate × Miss Penalty (Miss Penalty = Miss Cost)

Cache A (4KB): AMAT_A = 3 + 0.15 × 25 = 3 + 3.75 = 6.75 cycles

Cache B (12KB): AMAT_B = 6 + 0.05 × 25 = 6 + 1.25 = 7.25 cycles

Conclusion: Cache A (6.75 cycles) is better despite its smaller size. Lower hit cost more than compensates for higher miss rate.

Q8. Compose 4K×16 ROM using 1K×8 ROMs. Determine the better cache: Cache A: 2KB, Miss Rate=15%, Hit Cost=2, Miss Cost=30 Cache B: 8KB, Miss Rate=5%, Hit Cost=4, Miss Cost=30	**2076 Baisakh (Back) [5+3]**

ANSWER

Composing 4K×16 using 1K×8: (See Q5 above — 8 chips, 2-width×4-depth, A11–A10 as 2-to-4 bank-select.)

Cache AMAT Comparison:

Cache A: AMAT_A = 2 + 0.15 × 30 = 2 + 4.5 = 6.5 cycles

Cache B: AMAT_B = 4 + 0.05 × 30 = 4 + 1.5 = 5.5 cycles

Conclusion: Cache B (5.5 cycles) is better. Lower miss rate dominates despite higher hit cost.

Q9. Explain the cache mapping techniques. Compare direct mapping and fully associative mapping.	**2081 Chaitra (Regular) | 2079 Jestha (Back) | 2073 Magh (Regular) | 2076 Baisakh (Back) [4–8]**

ANSWER

Cache Mapping Techniques:

1. Direct Mapping: Each main memory block maps to exactly one cache line. Formula: Cache Line = Block Number mod Number_of_Cache_Lines. Address divided into: Tag | Line Index | Block Offset.

- Fast lookup (only 1 tag to check).
- Thrashing: two frequently used blocks mapping to the same line cause repeated evictions.

2. Fully Associative Mapping: A block can go into any cache line. Address: Tag | Block Offset. All tags checked in parallel (CAM).

- Best hit rate — no thrashing.
- Expensive hardware (parallel comparators for every line). Impractical for large caches.

3. Set-Associative Mapping (k-way): Cache divided into sets of k lines. Block maps to a specific set (direct-mapped within sets) but any line within the set (fully associative within set). Set = Block mod Num_Sets.

- Best compromise — 2-way, 4-way, 8-way are common in real processors.

Property	Direct	Fully Associative	Set Associative
Placement	1 specific line	Any line	Any line in a set
Tag search	1 compare	All lines	k lines in set
Hardware cost	Low	Very high	Moderate
Hit rate	Lower (thrashing)	Highest	High
Used in	Simple caches	TLBs, small caches	Most CPU L1/L2/L3 caches

Q10. Explain memory hierarchy. Explain cache memory. What are cache replacement algorithms?	**2081 Ashwin (Back) | 2080 Ashwin (Back) | 2079 Ashwin (Back) [3+3+2 / 4+4]**

ANSWER

Memory Hierarchy:

```
CPU Registers      Fastest | Smallest (~bytes-KB)  | Highest cost/bit
      ↕
L1 Cache (on-chip) Very fast (~2-5 ns)  | 16-64 KB
      ↕
L2 Cache (on-chip) Fast (~10-20 ns)     | 256 KB - 4 MB
      ↕
L3 Cache (on-chip) Moderate (~30-50 ns) | 4 - 32 MB
      ↕
Main Memory (DRAM) Slow (~50-70 ns)     | GBs
      ↕
Secondary Storage  Very slow (~ms)      | TBs   | Lowest cost/bit
```

- Temporal Locality: Recently accessed data likely accessed again soon (loops, functions).
- Spatial Locality: Data near recently accessed locations likely accessed soon (arrays, sequential code).

Cache Memory:

Cache is a small, fast SRAM memory between CPU and main RAM. When CPU accesses an address, the cache controller checks if the data is already cached (hit) or not (miss). On a miss, the block is fetched from RAM into cache.

Hit Rate = (Cache Hits) / (Total Accesses). AMAT = Hit Cost + Miss Rate × Miss Penalty.

Cache Replacement Algorithms:

LRU (Least Recently Used): Evict the line not accessed for the longest time. Best performance; needs access history tracking.

FIFO (First In First Out): Evict the oldest loaded line. Simple; ignores access frequency.

LFU (Least Frequently Used): Evict the line with fewest accesses. Can retain old rarely-used lines forever.

Random: Randomly evict a line. Surprisingly effective; avoids worst-case deterministic behavior.

OPT (Béládyʼs Algorithm): Evict the line not needed for the longest future time. Theoretical optimum — not implementable in practice.

Q11. Explain the data storing and erasing procedures used in EEPROM. Explain Flash memory.	2080 Ashwin (Back) [5+3]

ANSWER

EEPROM Data Storing and Erasing:

EEPROM uses floating-gate MOSFET transistors. Data is stored as charge trapped on the floating gate — alters threshold voltage.

Erasing: High voltage (~12–21V) applied to control gate. Fowler-Nordheim quantum tunneling removes electrons from floating gate through thin oxide → cell set to logic '1' (erased). Can erase individual bytes/pages electrically. Erase time: ~1–10 ms/byte. Endurance: ~100,000 cycles.

Writing (Programming): Target byte must be erased first (to '1'). Programming voltage injects electrons onto floating gate via tunneling → lowers threshold voltage → stores logic '0'. Write verification reads back the byte to confirm. Write time: ~1–5 ms/byte.

Flash Memory:

- Flash is a form of EEPROM that is erased in blocks (sectors/pages) rather than byte-by-byte — faster erase but less granularity.
- NOR Flash: Supports random byte read (execute-in-place, XIP). Used for program code storage in MCUs.
- NAND Flash: Block-only access. Higher density. Used in SD cards, eMMC, SSDs. Requires wear leveling.
- Both types have similar write/erase mechanisms (Fowler-Nordheim tunneling or hot-electron injection).

Q12. Design a ROM to store the following information (3 address inputs A,B,C — 4 output functions F1–F4): ABC=000:F1=0,F2=1,F3=1,F4=1 | 001:F1=0,F2=0,F3=1,F4=1 | 010:F1=0,F2=1,F3=1,F4=0 | 011:F1=0,F2=1,F3=0,F4=0 | 100:F1=1,F2=0,F3=0,F4=1 | 101:F1=1,F2=1,F3=1,F4=1 | 110:F1=1,F2=0,F3=0,F4=0 | 111:F1=0,F2=0,F3=1,F4=1

2078 Poush (Back) | 2074 Chaitra (Back) [3+5]

ANSWER

This is an 8×4 ROM (3 address inputs → 8 words, 4 output bits per word). A 3-to-8 decoder drives 8 word lines.

A	B	C	F1	F2	F3	F4
0	0	0	0	1	1	1
0	0	1	0	0	1	1
0	1	0	0	1	1	0
0	1	1	0	1	0	0
1	0	0	1	0	0	1
1	0	1	1	1	1	1
1	1	0	1	0	0	0
1	1	1	0	0	1	1

```
3-to-8 Address Decoder + Diode Matrix:

         F1   F2   F3   F4    (Bit Lines)
WL0:     0    1●   1●   1●
WL1:     0    0    1●   1●
WL2:     0    1●   1●   0
WL3:     0    1●   0    0
WL4:     1●   0    0    1●
WL5:     1●   1●   1●   1●
WL6:     1●   0    0    0
WL7:     0    0    1●   1●

● = connection present (stores '1')
  = no connection (stores '0')
Total connections = count of 1s = 18 diodes
```

Q13. A cache memory has the following parameters: Hit time = 5 ns, Miss penalty = 100 ns, Miss rate = 8%. Calculate AMAT. If the miss rate is reduced to 3% by doubling cache size (new hit time = 7 ns), which cache is better?

Model Question | 2077 Ashwin (Back) [4+4]

ANSWER

Cache A: Hit Time=5ns, Miss Rate=8%, Miss Penalty=100ns

AMAT_A = Hit Time + Miss Rate × Miss Penalty = 5 + 0.08 × 100 = 5 + 8 = 13 ns

Cache B: Hit Time=7ns, Miss Rate=3%, Miss Penalty=100ns

AMAT_B = 7 + 0.03 × 100 = 7 + 3 = 10 ns

Conclusion: Cache B (10 ns AMAT) is better than Cache A (13 ns). The reduction in miss rate from 8% to 3% more than compensates for the 2 ns increase in hit time.

General Rule for Cache Selection:

- If miss rate reduction × miss penalty > increase in hit cost → larger cache is better.
- Here: miss rate reduction benefit = (0.08 – 0.03) × 100 = 5 ns. Hit cost increase = 2 ns. Benefit (5 ns) > Cost (2 ns) → larger cache wins.

Q14. Differentiate between NOR Flash and NAND Flash memory. Where is each used in embedded systems?	**Model Question | 2076 Jestha (Back) [4+4]**

ANSWER

Property	NOR Flash	NAND Flash
Access type	Random byte read (like ROM)	Block/page access only
Execute in place (XIP)	Yes — code runs directly from Flash	No — must copy to RAM first
Read speed	Fast random read (~70–100 ns)	Slow random read, fast sequential
Write/Erase speed	Slower write/erase	Faster write/erase
Density	Lower (larger cell size)	Higher (smaller cell, denser)
Cost	Higher per bit	Lower per bit
Endurance	~100,000 erase cycles	~10,000–100,000 cycles (MLC lower)
Error correction	Not usually needed	Requires ECC (bit errors possible)
Typical use	MCU program storage, BIOS	SD cards, eMMC, SSDs, USB drives
Examples	STM32 internal Flash, BIOS ROM	Samsung eMMC, SanDisk SD card

Summary: Topics Covered in Chapter 4

S.N.	Topic	Exam(s) Asked	Marks
1	Write Ability & Storage Permanence (full table)	2082, 2079, 2078, 2075, 2074 (annual topic)	3+5
2	Types of Memory (ROM through SDRAM) + SRAM vs DRAM	2079, 2072 (multiple exams)	4–8
3	ROM Construction from Truth Table (2-address)	2080, 2075, 2074, 2070	3+5
4	ROM Construction (3-address, 8×4 ROM)	2078, 2074	3+5
5	Composing Memory — 2K×16 using 1K×4 (8 chips)	2082, 2081, 2078, 2075, 2073 (most common)	3–8
6	Composing Memory — 4K×16 using 1K×8 (8 chips)	2076, 2075	5+3
7	Composing Memory — 4K×32 using 2K×16	2076, Model Q	4+4
8	Composing 2^(k+1)×n / 2^(k+2)×n using 2^k×n	2081, 2080	4+4
9	Cache Mapping (Direct / Fully Assoc. / Set Assoc.)	2081, 2079, 2078, 2076, 2073 (annual)	4–8
10	AMAT Calculation / Cache Comparison	2078, 2076, 2077, Model Q	3–5
11	Memory Hierarchy (register→cache→RAM→disk)	2081, 2080	3–4
12	Cache Replacement Algorithms (LRU/FIFO/LFU/OPT)	2080, 2079	3
13	EEPROM Erase/Write Procedure	2080	5
14	Flash Memory — NOR vs NAND	2076, Model Q	4+4